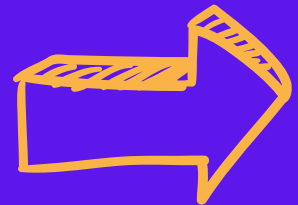


“Async Task” Or “Async Void”

Which one is correct ?

Farshad Fahimi





Async Task or Async Void

Farshad Fahimi

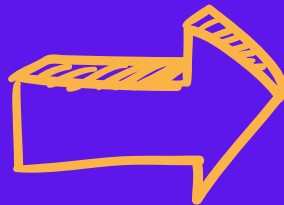
In general, we use asynchronous in c# with three following forms:

1) `async Task TestMethod() {}`

In this form of async calls, our method can be awaited but does not return any values.

2) `async Task<Person> TestMethod() {}`

In this form of async calls, our method can be awaited and also returns a value of type the Person.

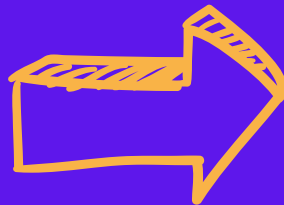




3) `async void TestMethod() {}`

In this form of async calls, which allows for fire and forget methods, our async method cannot be awaited.

The “await” keyword is used in an async method to inform the compiler that the method can have a suspension and resumption point.





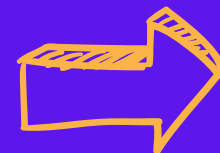
Async Task or Async Void

Farshad Fahimi

Async void methods exist for the single purpose of being used as an event handler. More specifically, it has been introduced to support UI elements event handlers, nothing else.

An async method with a void return type is a "fire and forget" method best reserved for event handlers because there's no way to wait for the method's execution to complete and respond accordingly. There's also no way to catch exceptions thrown from the method.

Having an async void method that is not an event handler could mean your program works sometimes and not others because of timing issues. Instead, async methods should return Task.





Async Task or Async Void

Farshad Fahimi

Here is an example of "**Async Void**" and why you should not use it in some situations, except those times, and you want to fire and forget something.

```
public class ClientService
{
    private string clientName;

    // Not recommended way to call a async process.
    public async void CallClientNameEndpoint(string url)
    {
        var client = new HttpClient();
        var response = await client.GetAsync(url);
        clientName = await response.Content.ReadAsStringAsync();
    }

    public async Task PrintClientName(string url)
    // works correctly if web request finishes in under 1 second, otherwise content will be null
    {
        CallClientNameEndpoint(url);
        await Task.Delay(1000);
        Console.Write(clientName);
    }
}
```



Async Task or Async Void

Farshad Fahimi

Here is an example of "**Async Task**" and why you should not use it in some situations, except those times, and you want to fire and forget something.

```
public class ClientService
{
    private string clientName;

    public async Task CallClientNameEndPoint(string url)
    {
        var client = new HttpClient();
        var response = await client.GetAsync(url);
        clientName = await response.Content.ReadAsStringAsync();
    }

    public async Task PrintClientName(string url)
    {
        //call changed here. If await is not added warning CS4014 will be triggered
        await CallClientNameEndPoint(url);
        await Task.Delay(1000);
        Console.Write(clientName);
    }
}
```



Async Task or Async Void

Farshad Fahimi

Be aware of the consequences of using “Async void” even if you wrapped it inside a try-catch block with the intent of handling the exception thrown inside the method. You be never noticed, and that is because the exception thrown inside the Method will never be caught.

TheEnd();

If you want to learn more,

Follow



Farshad Fahimi